Overview

Markdown is created by Daring Fireball, the original guideline is here. Its syntax, however, varies between different parsers or editors. **Typora** is using [GitHub Flavored Markdown][GFM].

Please note that HTML fragments in markdown source will be recognized but not parsed or rendered. Also, there may be small reformatting on the original markdown source code after saving.

Outline

- Overview
- Block Elements
  - Paragraph and line breaks
  - Headers
  - Blockquotes
  - Lists
    - un-ordered list
    - ordered list
    - Task List
  - (Fenced) Code Blocks
  - Tables
  - Footnotes
  - Horizontal Rules
  - YAML Front Matter
  - Table of Contents (TOC)
- Span Elements
  - Links
  - URLs
  - Images
  - Emphasis
  - Strong
  - Code
  - Strikethrough
  - Underline
  - Emoji 😊
  - HTML
- MathJax
  - Greek letters
  - Simple Syntax
- Diagrams (Sequence, Flowchart and Mermaid)
    - Sequence
    - Flowchart
      - Flowchart
      - Gantt
      - Class Diagram
      - State Diagram
      - Pie Chart
- REFERENCES

Block Elements

Paragraph and line breaks

A paragraph is simply one or more consecutive lines of text. In markdown source code, paragraphs are separated by more than one blank lines. In Typora, you only need to press `Return` to create a new paragraph.

Press `Shift` + `Return` to create a single line break. However, most markdown parser will ignore single line break, to make other markdown parsers recognize your line break, you can leave two whitespace at the end of the line, or insert `<br/>`.

Headers

Headers use 1-6 hash characters at the start of the line, corresponding to header levels 1-6. For example:

```
# This is an H1

## This is an H2

###### This is an H6
```

In typora, input `#`s followed by title content, and press `Return` key will create a header.

Blockquotes

Markdown uses email-style > characters for block quoting. They are presented as:

```
> This is a blockquote with two paragraphs. This is first paragraph.
>
> This is second pragraph.Vestibulum enim wisi, viverra nec, fringilla in,
laoreet vitae, risus.
```

> This is another blockquote with one paragraph. There is three empty line to seperate two blockquote.

In typora, just input `>` followed by quote contents a block quote is generated. Typora will insert proper '>' or line break for you.

Lists

Input `* list item 1` will create an un-ordered list, the `*` symbol can be replace with `+` or `-`.

Input `1. list item 1` will create an ordered list, their markdown source code is like:

un-ordered list

```
*   Red
*   Green
*   Blue
```

- Red
- Green
- Blue

ordered list

```
1. Red
2. GreenBlue
3. Yellow
```

1. Red
2. GreenBlue
3. Yellow

Task List

Task lists are lists with items marked as either [ ] or [x] (incomplete or complete). For example:

```
- [ ] a task list item
- [ ] list syntax required
- [ ] normal **formatting**
- [ ] incomplete
- [x] completed
```

- [ ] a task list item
- [ ] list syntax required
- [ ] normal **formatting**
- [ ] incomplete
- [ ] completed

You can change the complete/incomplete state by click the checkbox before the item.

(Fenced) Code Blocks

Typora only support fences in GFM. Original code blocks in markdown is not supported.

Using fences is easy: Input ```` ```` and press `return`. Add an optional language identifier after ```` ```` and we'll run it through syntax highlighting:

Here's an example:

```
function test() {
  console.log("notice the blank line before this function?");
}
```

syntax highlighting:

```
require 'redcarpet'
markdown = Redcarpet.new("Hello World!")
puts markdown.to_html
```

Tables

Input `| First Header | Second Header |` and press `return` key will create a table with two column.

After table is created, focus on that table will pop up a toolbar for table, where you can resize, align, or delete table. You can also use context menu to copy and add/delete column/row.

Following descriptions can be skipped, as markdown source code for tables are generated by typora automatically.

In markdown source code, they look like:

```
| First Header | Second Header |
| ------------ | ------------- |
| Content Cell | Content Cell  |
| Content Cell | Content Cell  |
```

You can also include inline Markdown such as links, bold, italics, or strikethrough.

Finally, by including colons : within the header row, you can define text to be left-aligned, right-aligned, or center-aligned:

```
| Left-Aligned  | Center Aligned  | Right Aligned |
| :------------ | :-------------: | ------------: |
| col 3 is      | some wordy text |         $1600 |
| col 2 is      |    centered     |           $12 |
| zebra stripes |    are neat     |            $1 |
```

A colon on the left-most side indicates a left-aligned column; a colon on the right-most side indicates a right-aligned column; a colon on both sides indicates a center-aligned column.

Footnotes

```
You can create footnotes like this[^footnote].

[^footnote]: Here is the *text* of the **footnote**.
```

will produce:

You can create footnotes like this [1].

Mouse on the 'footnote' superscript to see content of the footnote.

Horizontal Rules

Input `***` or `---` on a blank line and press `return` will draw a horizontal line.

YAML Front Matter

Typora support YAML Front Matter now. Input `---` at the top of the article and then press `Enter` will introduce one. Or insert one metadata block from the menu.

Table of Contents (TOC)

Input `[toc]` then press `Return` key will create a section for "Table of Contents" extracting all headers from one's writing, its contents will be updated automatically.

Span Elements

Span elements will be parsed and rendered right after your typing. Moving cursor in middle of those span elements will expand those elements into markdown source. Following will explain the syntax of those span element.

Links

Markdown supports two style of links: inline and reference.

In both styles, the link text is delimited by [square brackets].

To create an inline link, use a set of regular parentheses immediately after the link text's closing square bracket. Inside the parentheses, put the URL where you want the link to point, along with an optional title for the link, surrounded in quotes. For example:

```
This is [an example](http://example.com/ "Title") inline link.

[This link](http://example.net/) has no title attribute.
```

will produce:

This is an example inline link. (`<p>This is <a href="http://example.com/" title="Title">`)

This link has no title attribute. (`<p><a href="http://example.net/">This link</a> has no`)

You can set the href to headers, which will create a bookmark that allow you to jump to that section after clicking. For example:

Command(on Windows: Ctrl) + Click This link [2] will jump to header `Block Elements`. To see how to write that, please move cursor or click that link with `⌘` key pressed to expand the element into markdown source.

Reference-style links use a second set of square brackets, inside which you place a label of your choosing to identify the link:

This is an example reference-style link.

Then, anywhere in the document, you define your link label like this, on a line by itself:

```
[id]: http://example.com/  "Optional Title Here"
```

In typora, they will be rendered like:

This is an example reference-style link.

The implicit link name shortcut allows you to omit the name of the link, in which case the link text itself is used as the name. Just use an empty set of square brackets — e.g., to link the word "Google" to the google.com web site, you could simply write:

Google And then define the link:

In typora click link will expand it for editing, `command+click` will open the hyperlink in web browser.

URLs

Typora allows you to insert urls as links, wrapped by `<`brackets`>`.

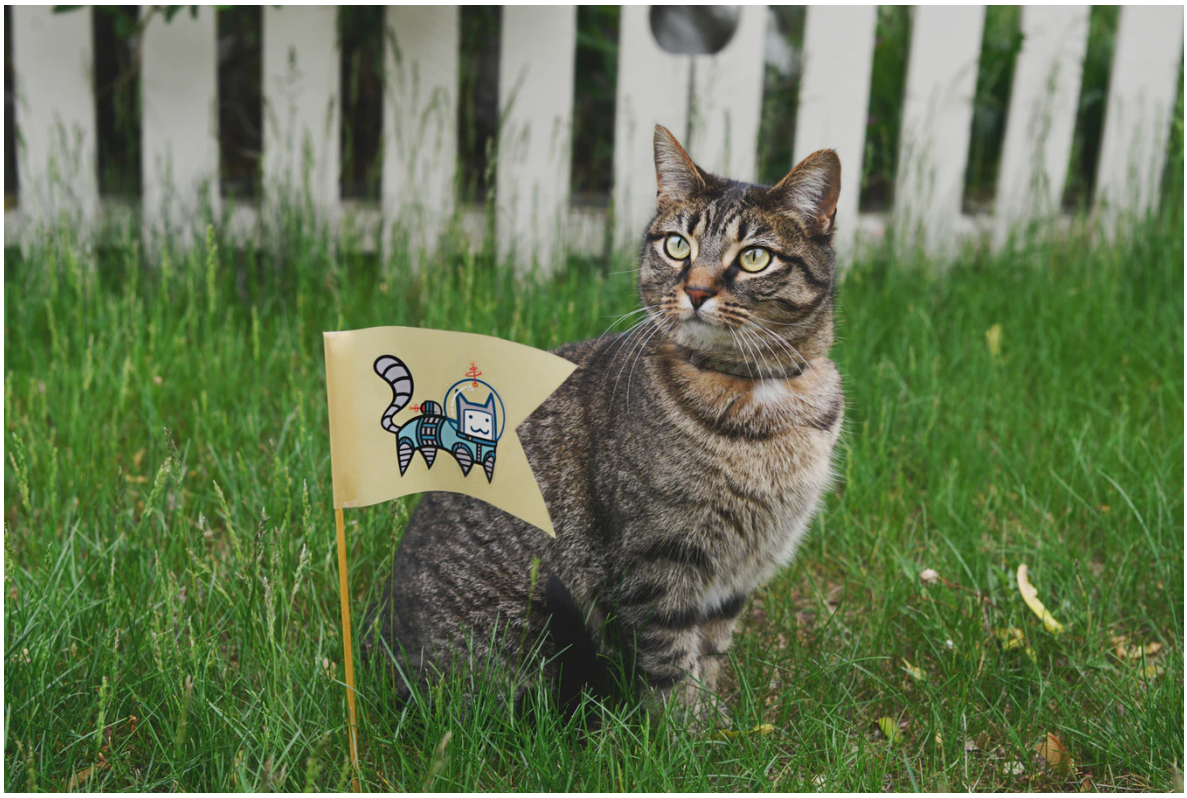
`<i@typora.io>` becomes i@typora.io.

Typora will aslo auto link standard URLs. e.g: www.google.com.

Images

Image looks similar with links, but it requires an additional `!` char before the start of link. Image syntax looks like this:

```
![This is a cat](https://images.unsplash.com/photo-1484244233201-29892afe6a2c?ixlib=rb-0.3.5&ixid=eyJhcHBfaWQiOjEyMDd9&s=2db8de5b9f0478ba59dfd9d61f4d64fe&auto=format&fit=crop&w=1350&q=80)
```



Emphasis

Markdown treats asterisks (`*`) and underscores (`_`) as indicators of emphasis. Text wrapped with one `*` or `_` will be wrapped with an HTML `<em>` tag. E.g:

single asterisks

single underscores

GFM will ignores underscores in words, which is commonly used in code and names, like this:

> wow_great_stuff
>
> do_this_and_do_that_and_another_thing.

To produce a literal asterisk or underscore at a position where it would otherwise be used as an emphasis delimiter, you can backslash escape it:

`\*this text is surrounded by literal asterisks\*`

Markdown provides backslash escapes for the following characters:

```
\   backslash
`   backtick
*   asterisk
_   underscore
{}  curly braces
[]  square brackets
()  parentheses
#   hash mark
+   plus sign
-   minus sign (hyphen)
.   dot
!   exclamation mark
```

Typora recommends to use `*` symbol.

Strong

double *'s or _'s will be wrapped with an HTML `<strong>` tag, e.g:

```
**double asterisks**

__double underscores__
```

double asterisks

double underscores

Typora recommends to use `**` symbol.

Code

To indicate a span of code, wrap it with backtick quotes (`). Unlike a pre-formatted code block, a code span indicates code within a normal paragraph. For example:

Use the `printf()` function.

Strikethrough

GFM adds syntax to create strikethrough text, which is missing from standard Markdown.

`~~Mistaken text.~~` becomes ~~Mistaken text.~~

Underline

Underline is powered by raw HTML.

`<u>Underline</u>` becomes <u>Underline</u>.

Emoji 😄

Input emoji with syntax `:smile:`.

User can trigger auto-complete suggestions for emoji by pressing `ESC` key, or trigger it automatically after enable it on preference panel. Also, input UTF8 emoji char directly from `Edit` -> `Emoji & Symbols` from menu bar is also supported.

HTML

Typora cannot render html fragments. But typora can parse and render very limited HTML fragments, as an extension of Markdown, including:

- Underline: `<u>underline</u>`
- Image: `<img src="http://www.w3.org/html/logo/img/mark-word-icon.png" width="200px" />` (And `width`, `height` attribute in HTML tag, and `width`, `height`, `zoom` style in `style` attribute will be applied.)

- Comments: `<!-- This is some comments -->`
- Hyperlink: `<a href="http://typora.io" target="_blank">link</a>`. link

Most of their attributes, styles, or classes will be ignored. For other tags, typora will render them as raw HTML snippets.

MathJax

But those HTML will be exported on print or export. You can render *LaTeX* mathematical expressions using MathJax, Check More From MathJaxTexSyntax

Greek letters

LETTERS	letters	TEX	tex
A	α	`A`	`\alpha`
B	β	`B`	`\beta`
Γ	γ	`\Gamma`	`\gamma`
Δ	δ	`\Delta`	`\delta`
E	ϵ	`E`	`\epsilon`
Z	ζ	`Z`	`\zeta`
H	η	`H`	`\eta`
Θ	θ	`\Theta`	`\theta`
I	ι	`I`	`\iota`
k	κ	`k`	`\kappa`
Λ	λ	`\Lambda`	`\lambda`
M	μ	`M`	`\mu`
N	ν	`N`	`\nu`
Ξ	ξ	`\Xi`	`\xi`
O	o	`O`	`\omicron`
Π	π	`\Pi`	`\pi`
P	ρ	`P`	`\rho`
Σ	σ	`\Sigma`	`\sigma`
T	τ	`\Sigma`	`\tau`
Υ	υ	`\Upsilon`	`\upsilon`
Φ	ϕ	`\Phi`	`\phi`
X	χ	`X`	`\chi`
Ψ	ψ	`\Psi`	`\psi`
Ω	ω	`\Omega`	`\omega`

Simple Syntax

`\mathcal{ ABCDEFGHIJKLMNOPQRSTUVWXYZ}`: $\mathcal{ABCDEFGHIJKLMNOPQRSTUVWXYZ}$
`\mathscr{ABCDEFGHIJKLMNOPQRSTUVWXYZ}`:
$\mathscr{ABCDEFGHIJKLMNOPQRSTUVWXYZ}$
`\mathsf{ABCDEFGHIJKLMNOPQRSTUVWXYZ}`: ABCDEFGHIJKLMNOPQRSTUVWXYZ

`$ q \in R $` : $q \in R$

`$$ q \in R $$` : $q \in R$

`\cup \cap \setminus \subset \subseteq \subsetneq \supset \in \notin \emptyset \varnothing` : $\cup \cap \setminus \subset \subseteq \subsetneq \supset \in \notin \emptyset \varnothing$

`() [] \{ \}\langle x \rangle \lceil x \rceil \lfloor x \rfloor` :

$() [] \{ \} \langle x \rangle \lceil x \rceil \lfloor x \rfloor$

`\int_1^\infty \prod \bigcup \bigcap \iint \iiint` : $\int_1^\infty \prod \bigcup \bigcap \iint \iiint$

`a_1+a_2+\cdots+a_n` : $a_1 + a_2 + \cdots + a_n$

`x_i^2` : x_i^2 `\log_2 x` : $\log_2 x$ `2^{16}` : 2^{16}

`\lim_{x \to \infty} \exp(-x) = 0` : $\lim_{x \to \infty} \exp(-x) = 0$

`\left(\frac{\sqrt [3] \frac {x}{y}}{y^3}\right)` : $\left(\frac{\sqrt[3]{\frac{x}{y}}}{y^3}\right)$

`$$\sum_{i=0}^n i^2 = \frac{(n^2+n)(2n+1)}{6}$$` : $\sum_{i=0}^n i^2 = \frac{(n^2+n)(2n+1)}{6}$

```
$$
\mathbf{V}_1 \times \mathbf{V}_2 =  \begin{vmatrix}
\mathbf{i} & \mathbf{j} & \mathbf{k} \\\\
\frac{\partial X}{\partial u} &  \frac{\partial Y}{\partial u} & 0 \\\\
\frac{\partial X}{\partial v} &  \frac{\partial Y}{\partial v} & 0 \\\\
\end{vmatrix}
$$
```

$$
\mathbf{V}_1 \times \mathbf{V}_2 = \begin{vmatrix}
\mathbf{i} & \mathbf{j} & \mathbf{k} \\
\frac{\partial X}{\partial u} & \frac{\partial Y}{\partial u} & 0 \\
\frac{\partial X}{\partial v} & \frac{\partial Y}{\partial v} & 0 \\
\end{vmatrix}
$$

Diagrams (Sequence, Flowchart and Mermaid)

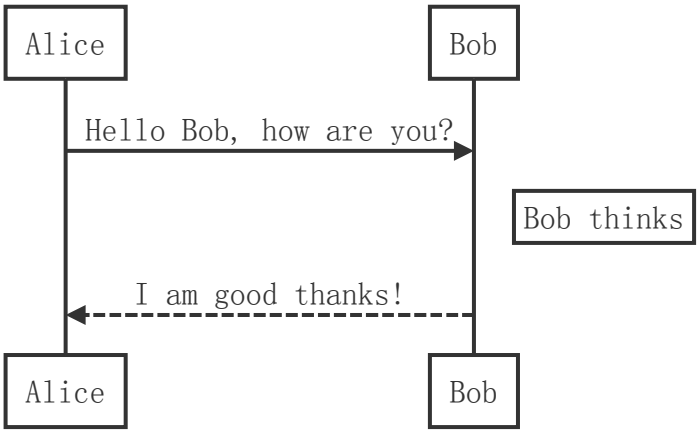
Typora supports, sequence, flowchart and mermaid, after this feature is enabled from preference panel.

> See this document for detail.
>
> When exporting as HTML, PDF, epub, docx, those rendered diagrams will also be included, but diagrams features are not supported when exporting markdown into other file formats in current version. Besides, you should also notice that diagrams is not supported by standard Markdown, CommonMark or GFM. Therefore, we still recommend you to insert an image of these diagrams instead of write them in Markdown directly.

It is powered by js-sequence, which would turn following code block into rendered diagrams:

Sequence

```
```sequence
Alice->Bob: Hello Bob, how are you?
Note right of Bob: Bob thinks
Bob-->Alice: I am good thanks!
```
```

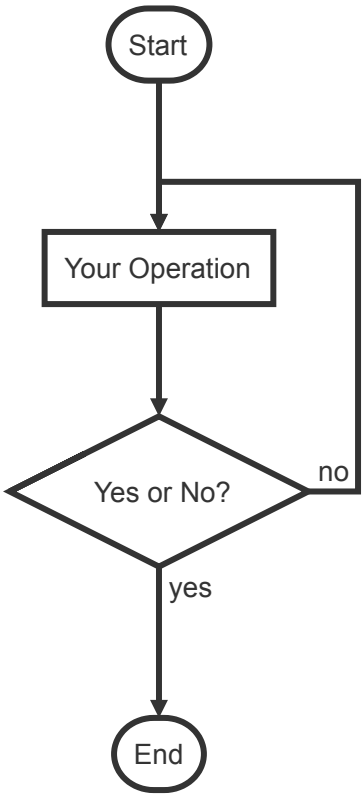


Please refer here for syntax explanation.

Flowchart

It is powered by flowchart.js, which would turn following code block into rendered diagrams:

~gfm



```flow
st=>start: Start
op=>operation: Your Operation
cond=>condition: Yes or No?
e=>end

st->op->cond
cond(yes)->e
cond(no)->op
```

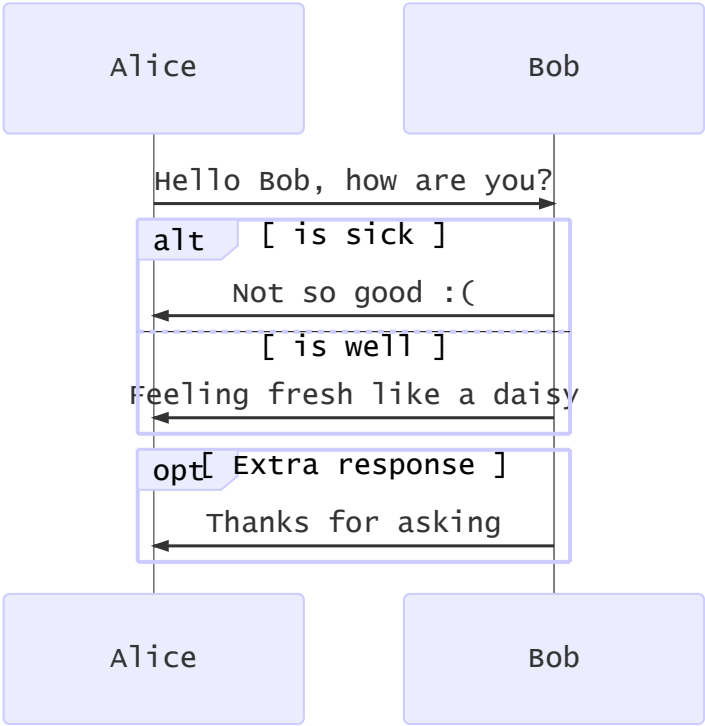
Mermaid

Typora also has integration with [mermaid](https://knsv.github.io/mermaid/#/), which supports sequence, flowchart and Gantt.

Sequence

see [this doc](https://mermaid-js.github.io/mermaid/#/sequenceDiagram)

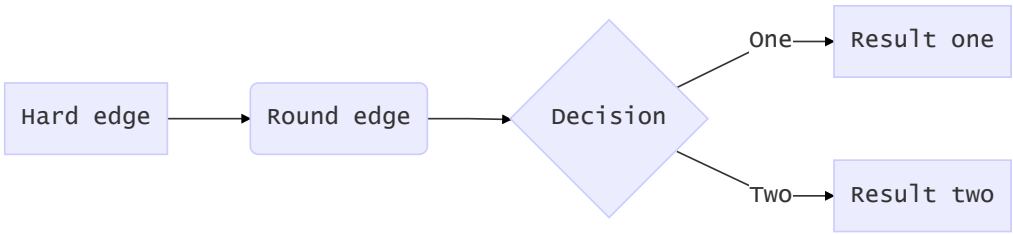
~~~gfm
```mermaid
%% Example of sequence diagram
  sequenceDiagram
    Alice->>Bob: Hello Bob, how are you?
    alt is sick
    Bob->>Alice: Not so good :(
    else is well
    Bob->>Alice: Feeling fresh like a daisy
    end
    opt Extra response
    Bob->>Alice: Thanks for asking
    end
```
~~~



Flowchart

see this doc

```mermaid
graph LR
A[Hard edge] -->B(Round edge)
    B --> C{Decision}
    C -->|One| D[Result one]
    C -->|Two| E[Result two]
```



Gantt

see this doc

```mermaid
%% Example with selection of syntaxes
        gantt
        dateFormat  YYYY-MM-DD
```

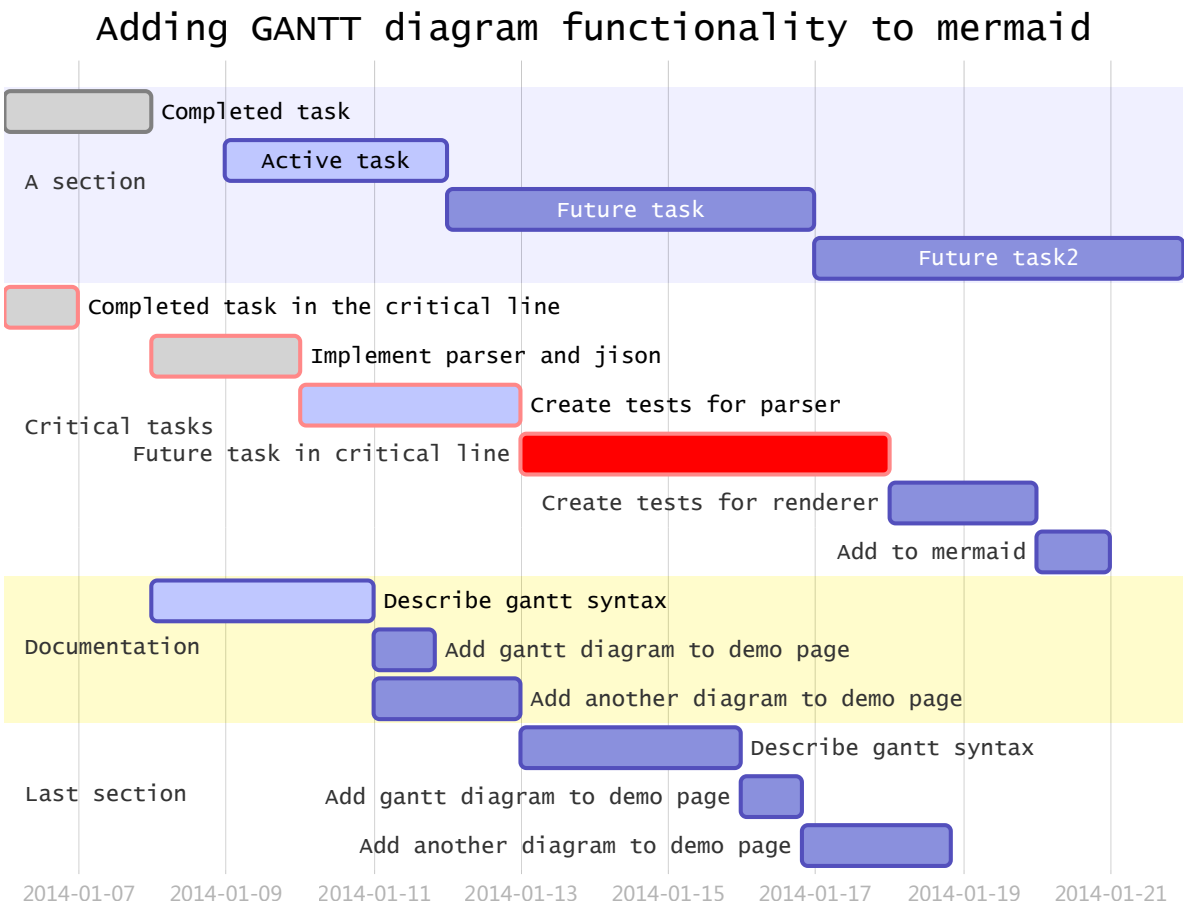
```
        title Adding GANTT diagram functionality to mermaid

        section A section
        Completed task            :done,    des1, 2014-01-06,2014-01-08
        Active task               :active,  des2, 2014-01-09, 3d
        Future task               :         des3, after des2, 5d
        Future task2               :         des4, after des3, 5d

        section Critical tasks
        Completed task in the critical line :crit, done, 2014-01-06,24h
        Implement parser and jison          :crit, done, after des1, 2d
        Create tests for parser             :crit, active, 3d
        Future task in critical line        :crit, 5d
        Create tests for renderer           :2d
        Add to mermaid                      :1d

        section Documentation
        Describe gantt syntax               :active, a1, after des1, 3d
        Add gantt diagram to demo page      :after a1  , 20h
        Add another diagram to demo page    :doc1, after a1  , 48h

        section Last section
        Describe gantt syntax               :after doc1, 3d
        Add gantt diagram to demo page      : 20h
        Add another diagram to demo page    : 48h
```



Class Diagram

see this doc

```mermaid
classDiagram
      Animal <|-- Duck
      Animal <|-- Fish
      Animal <|-- Zebra
      Animal : +int age
      Animal : +String gender
      Animal: +isMammal()
      Animal: +mate()
      class Duck{
          +String beakColor
          +swim()
          +quack()
      }
      class Fish{
          -int sizeInFeet
          -canEat()
      }
      class Zebra{
          +bool is_wild
          +run()
      }
```

State Diagram

see this doc

```mermaid
stateDiagram
    [*] --> Still
    Still --> [*]

    Still --> Moving
    Moving --> Still
    Moving --> Crash
    Crash --> [*]
```

Pie Chart

```mermaid
pie
    title Pie Chart
    "Dogs" : 386
    "Cats" : 85
    "Rats" : 150
```

REFERENCES

- http://support.typora.io
- http://www.onemathematicalcat.org/MathJaxDocumentation/TeXSyntax.htm

---

1. Here is the *text* of the **footnote**.↩

2. [This link](#block-elements)↩